

Assignment Project V2 — Development Plan

Olist Brazilian E-Commerce on Google BigQuery

Group 7

Prepared: 1 September 2026

Repository: github.com/PeterLees/bigData-module2-project-group7

STATUS: FOR TEAM REVIEW — NOT YET APPROVED FOR DEVELOPMENT

This document is the plan of record. It fixes the dataset, the warehouse region, the schema grain, the tool choices, the ownership model, the quality gates and the phase-by-phase schedule. **Nothing is built until every team member has reviewed Sections 2, 4, 6 and 8 and signed off in Section 16.**

Contents

1. Context and Grading Logic
2. Locked Decisions
3. Target Architecture
4. Data Model: Star Schema
5. Repository Structure
6. Ownership and Access Control
7. Manual Setup Steps
8. Phased Delivery Plan
9. Data Quality Matrix
10. Analysis Plan
11. Orchestration and CI
12. Coverage of the Assignment Brief
13. Executive Presentation Structure
14. Risks and Mitigations
15. Final Acceptance Checklist
16. Review and Sign-off

1. Context and Grading Logic

The repository currently contains only the assignment brief and the two course guides. There is no code. This plan turns the Module 2 Assignment Project V2 brief into an executable engineering programme: ingest a raw dataset, model it as a star schema in a cloud warehouse, transform it with an ELT tool, prove data quality with automated tests, analyse it in Python, orchestrate the whole thing, and present it to a mixed executive audience.

Both course guides converge on the same grading logic, and this plan is built around it:

- Marks come from the evidence chain, not from the tool count. Source and contract, then raw load, then transformation lineage, then quality gates, then optimised query, then insight, then recommendation, then caveat.
- Roughly 80% of effort goes to the pipeline, architecture, access control, quality and reproducibility. Roughly 20% goes to a small number of decisive analyses. One decisive chart beats ten disconnected ones.
- Complexity only earns credit when it solves a stated constraint and leaves operational evidence. Kafka, Spark and Redis are deliberately out of scope; they appear only in the future-expansion section, with the threshold that would justify them.
- Every table must have a declared grain, a primary key, an owner and a description before any SQL is written.

1.1 The primary business case

BUSINESS CASE 3 — DELIVERY PERFORMANCE. Management question: where and when are deliveries late? The course guide is explicit that a strong project picks one defensible business case early and lets it determine the grain, the dimensions, the tests and the final chart. This is ours. Every design decision below is traceable to it.

The business case decomposes into four questions we must be able to answer with evidence:

#	Question	Why an executive cares
D1	WHERE are deliveries late? Which customer states, which seller states, and which routes between them?	Determines which carrier contracts, regional hubs or seller SLAs to renegotiate first.
D2	WHEN are deliveries late? Which months, weeks and days of week, and is the delay trend improving or worsening?	Determines seasonal capacity planning and whether past logistics investment worked.
D3	WHAT does lateness cost us? Its association with review score, and the revenue exposed to late delivery.	Converts a logistics metric into money and satisfaction, which is what a CFO or COO will act on.
D4	HOW BAD is the estimate itself? Is the promised delivery date systematically wrong, and by how much?	A promise nobody can keep destroys trust even when the actual delivery time is acceptable. Fixing the promise is often cheaper than fixing the logistics.

1.2 Supporting analyses required by the brief

Section 5 of the assignment brief separately mandates monthly sales trends, top-selling products and customer segmentation. These are delivered in full, and each is framed through the delivery lens so the report tells one story rather than four:

- Monthly sales trend, cross-read against the monthly late-delivery rate, so revenue and service quality are seen together.

- Top products and categories, with their delivery-delay and review profile attached, so we can identify categories that sell well but ship badly.
- Customer segmentation by RFM, cross-tabulated with delivery experience, so we can quantify how much high-value customer revenue is exposed to late delivery.

2. Locked Decisions

These are settled. Changing any of them after Phase 0 is expensive, in particular the BigQuery location, which cannot be altered after a dataset is created. Raise objections during this review, not later.

Decision	Choice	Rationale and rejected alternative
Dataset	Olist Brazilian E-Commerce (Kaggle: olistbr/brazilian-ecommerce)	The course guide default recommendation. Nine related tables, about 1.55M rows, and real money columns, so it supports multi-table star modelling, revenue reconciliation, RFM segmentation and delivery-performance analysis. Rejected: Instacart (34M rows but no money columns, so no revenue KPI); London Bicycles (forces an EU-only project and has no customer entity, which makes the required customer-segmentation analysis impossible).
Warehouse	Google BigQuery, location US multi-region	The stack taught in units 2.5 to 2.7. Olist carries no data-residency constraint, so US multi-region gives the widest compatibility, keeps the always-free tier (1 TB scanned and 10 GB storage per month) and allows joins to US-hosted public datasets. The GCS bucket and all BigQuery datasets are pinned to the same location to avoid cross-region failures and egress cost.
Ingestion	Python script (google-cloud-storage plus google-cloud-bigquery) with an explicit manifest	Olist is a static CSV export, so Meltano incremental state would add configuration without solving a problem. A Python loader with a checksum manifest and WRITE_TRUNCATE gives idempotency and a reconciliation report in far less code. Meltano is documented as the migration path if a live source is added.
Transformation	dbt-core plus dbt-bigquery (ELT, not ETL)	Raw data stays immutable and replayable in BigQuery; transformation is version-controlled SQL and YAML that can be reviewed in a pull request; lineage, tests and documentation are generated from the same models.
Quality	dbt tests (in-warehouse contracts) plus Great Expectations (pre-load file gate)	GX earns its place by validating CSV files before they reach BigQuery, which dbt cannot do. dbt tests own everything that already lives in a table. This is a deliberate division of responsibility, not tool collecting.
Orchestration	Dagster asset DAG and schedule; GitHub Actions for CI	Item 6 of the brief is optional; we take it because it is where operational evidence comes from. Dagster gives asset lineage, retry policy and a demonstrable fail-stop; GitHub Actions gives pull-request-time evidence. A bare cron job is explicitly not orchestration.
Analysis	SQLAlchemy plus pandas in Jupyter	Required verbatim by the brief. Notebooks read from the marts layer only, never from a manually uploaded CSV.
Team	Five or more members, one shared GCP project	Requires the explicit IAM and ownership model in Section 6 so that members cannot overwrite each other tables.

3. Target Architecture

Data flows in one direction through the layers below. Each numbered step is something a person can rerun on its own, and each shaded gate is a place where bad data stops rather than continuing downstream.

1 SOURCE	Kaggle Olist export. Nine CSV files, about 120 MB, 1,550,922 rows. Optionally enriched with Brazilian public holidays from BrasilAPI.
2 INGEST	ingestion/load_olist.py. Builds a manifest holding the sha256, byte size and true CSV record count of every file.
3 LANDING	Google Cloud Storage. gs://<PROJECT_ID>-olist-raw/olist/raw/v1/ - an immutable, replayable copy of exactly what was downloaded.
GATE 1	Great Expectations, on the files, before the load. Expected files present, column set and order, row count against the manifest, non-null keys, money ranges, categorical domains. On failure nothing is loaded at all.
4 RAW	BigQuery olist_raw.* Explicit schemas, autodetect disabled, source fields preserved as-is, plus _batch_id, _loaded_at, _source_uri and _source_sha256.
5 STAGING	dbt stg_* models. Rename, cast, deduplicate, translate categories, and collapse geolocation to one row per zip prefix.
6 INTERMEDIATE	dbt int_* models - the fan-out controls. Payments aggregated to order grain, reviews reduced to one per order, item lines rolled up, and the delivery milestone arithmetic.
7 MARTS	dbt dim_*, fct_* and agg_* models. The consumption contract: three facts, four dimensions and five aggregates, partitioned and clustered.
GATE 2	dbt build - 161 tests, interleaved with the models. Uniqueness, not-null, referential integrity, accepted values, numeric ranges, milestone chronology and source-to-mart reconciliation. On failure the marts do not publish.
8 SERVING	Notebooks, dashboard, report and deck. Jupyter via SQLAlchemy, a Streamlit dashboard, the written report and the executive deck - all reading the marts layer only.
ACROSS ALL	Orchestration and CI. A Dagster asset DAG with a daily schedule runs steps 1 to 7 in dependency order and stops downstream work when a gate fails. GitHub Actions checks every pull request.

3.1 Layer responsibilities and required evidence

Layer	Implementation	Evidence we must retain
Source	Kaggle Olist export, 9 CSV files, about 120 MB	Source URL, licence (CC BY-NC-SA 4.0), snapshot date, file list with sizes and sha256
Landing	GCS bucket, versioned prefix olist/raw/v1/	Object listing, upload log, replay instructions
Raw	BigQuery dataset olist_raw, source fields preserved as-is	Explicit JSON schemas, load job log, row-count reconciliation against the manifest
Staging	dbt stg_* models, one per source table, no cross-process joins	Compiled SQL, column-level documentation, source and staging tests
Intermediate	dbt int_* models, reusable business logic, fan-out elimination	Lineage graph, grain statement, fan-out check queries returning zero

Layer	Implementation	Evidence we must retain
Marts	dbt dim_*, fct_*, and agg_* models. Aggregates: agg_delivery_by_route, agg_delivery_monthly, agg_monthly_sales, agg_product_performance, agg_customer_rfm	Star-schema ERD, primary and foreign keys, metric dictionary, partition and cluster configuration
Quality	GX checkpoint before load; dbt build after every model	PASS/FAIL artifacts, failing-row samples, the fault-injection before and after
Serving	Jupyter via SQLAlchemy; report; executive deck	Reproducible notebooks, chart sources, insight cards
Orchestration	Dagster assets and schedule; GitHub Actions CI	DAG screenshot, run logs, a failed-upstream run showing skipped downstream

3.2 Deliberately out of scope

Named here so the team can answer the obvious Q&A question without hesitating.

Technology	Why not now	What would trigger adopting it
Apache Spark	1.55M rows and about 120 MB fit comfortably in BigQuery and in single-machine memory. A cluster adds scheduling, shuffle and operations cost for no gain.	Working set exceeds single-node memory, or a job misses its SLA on BigQuery with the scan already optimised.
Kafka and Structured Streaming	The dataset is a static historical export. There is no event stream to consume and no business value that decays with latency.	A live order feed exists and a decision (fraud hold, courier dispatch) loses value within minutes.
Redis	No low-latency serving surface in this project. A cache is not a source of truth.	A customer-facing API needs sub-10 ms reads of a precomputed segment.
Meltano	Static CSV needs no incremental state, catalog or bookmark management.	A continuously growing API or database replica is added as a second source.

4. Data Model: Star Schema

GRAIN FIRST. The single rule that governs this section: write down what one row of each fact table represents before writing any SQL. The grain determines the primary key, which measures are additive, and which joins are safe. Every documented Olist failure comes from skipping this step.

Because the primary business case is delivery performance, `fct_orders` is the headline fact table: lateness is an order-level property, not an item-level one. `fct_order_items` remains the revenue backbone, so that money questions and delivery questions can be answered from the same warehouse without mixing grains.

4.1 Grain declarations

Model	One row represents	Key	Measures and notes
<code>fct_order_items</code>	One item line within one order	<code>order_item_key = order_id '-' order_item_id</code>	<code>item_price</code> , <code>freight_value</code> , <code>item_gross_value</code> . Partitioned by <code>order_purchase_date</code> ; clustered by <code>customer_key</code> , <code>product_key</code> , <code>seller_key</code> .
<code>fct_payments</code>	One payment record	<code>payment_key = order_id '-' payment_sequential</code>	<code>payment_value</code> , <code>payment_installments</code> , <code>payment_type</code> . Kept as a separate fact so it can never multiply item revenue.
<code>fct_orders</code> (PRIMARY FACT for this business case)	One order, modelled as an accumulating snapshot across its delivery milestones	<code>order_id</code>	Milestones: <code>purchased</code> , <code>approved</code> , <code>handed to carrier</code> , <code>delivered</code> , <code>estimated</code> . Delivery measures: <code>approval_days</code> , <code>seller_handover_days</code> , <code>carrier_transit_days</code> , <code>total_delivery_days</code> , <code>promised_delivery_days</code> , <code>delay_vs_promise_days</code> , <code>promise_slack_days</code> , <code>is_late</code> (boolean), <code>lateness_bucket</code> . Money: <code>item_count</code> , <code>order_revenue</code> , <code>order_freight</code> , <code>order_payment_total</code> . Experience: <code>review_score</code> .
<code>dim_customer</code>	One person	<code>customer_key</code> (from <code>customer_unique_id</code>)	<code>customer_state</code> and <code>customer_city</code> are first-class here: they are the WHERE of the business case. Also zip prefix, region grouping, first and latest order date. SCD Type 1.
<code>dim_product</code>	One product	<code>product_key</code> (<code>product_id</code>)	English category name, weight and dimensions (both plausible drivers of transit time), photo count. SCD Type 1 plus a dbt snapshot for Type 2 evidence.

Model	One row represents	Key	Measures and notes
dim_seller	One seller	seller_key (seller_id)	seller_state and seller_city are first-class: origin is half of a delivery route. Also zip prefix and region grouping. SCD Type 1.
dim_date	One calendar day, 2016-01-01 to 2018-12-31	date_key	Day, week, month, quarter, year, weekday flag, Brazilian public-holiday flag if the optional enrichment is built.

4.2 Fan-out controls: the four Olist traps

These are the specific, documented ways this dataset silently produces wrong numbers. Each has a countermeasure and a test.

Trap	What goes wrong	Countermeasure
Payment fan-out	Joining raw payments to order items multiplies rows: N items times M payment lines. Revenue inflates, sometimes by 2 to 3 times, and the error looks plausible.	Aggregate payments to order grain in int_order_payments before any join. Enforced by a reconciliation test: sum(fct_order_items.item_price) must equal sum(stg_order_items.price).
Review fan-out	olist_order_reviews contains repeated review_id values and some orders with more than one review.	Deduplicate to one review per order in int_order_reviews (latest review_creation_date, tie-broken on review_id). Enforced by a unique test on order_id.
Customer identity	customer_id is issued per order, not per person. Using it for RFM splits one repeat buyer into several one-time customers and destroys the segmentation.	dim_customer is built on customer_unique_id. customer_id is retained only as a degenerate attribute on the order. Enforced by a unique test on customer_key.
Geolocation cardinality	olist_geolocation has about 1M rows and many rows per zip-code prefix. Joining it directly to customers or sellers explodes the row count.	Collapse to one row per zip prefix in stg_geolocation (median latitude and longitude, modal city and state) before it enriches any dimension.

4.3 Physical design and the cost benchmark

- fct_order_items and fct_orders are partitioned by order_purchase_date (DATE) and clustered by customer_key, product_key, seller_key, the three columns our analyses filter and join on most.
- Queries filter the partition column directly and never wrap it in a function, so partition pruning can occur. SELECT * is banned in models and notebooks.
- Deliverable evidence:** a three-way benchmark of the same business query against an unpartitioned copy, a partitioned copy, and a partitioned and clustered copy, recording bytes processed, duration and slot-milliseconds, with the result cache disabled. This is the optimisation proof the course guide asks for and the answer to "how do you control BigQuery cost?"

4.4 Slowly changing dimensions, stated honestly

Olist is a static historical export, so no dimension actually changes over the life of the project. dim_customer, dim_product and dim_seller are therefore SCD Type 1, and we say so. One dbt snapshot is built on product category

to demonstrate Type 2 mechanics (valid_from, valid_to, current flag) as course evidence. We will describe it as a demonstration of capability, not as a pretence that history is being captured.

4.5 How the business case drives every model decision

This is the table to point at in Q&A when asked "why is your schema shaped like that?"

Business-case need	Model decision it forces	Consequence if we got it wrong
Lateness is a property of an order, not of an item line	fct_orders is an accumulating snapshot at order grain, holding every milestone and every derived duration. Delivery measures never live on fct_order_items.	Averaging delay across item lines would silently weight multi-item orders more heavily and skew every regional figure.
"WHERE" means both destination and origin	dim_customer and dim_seller both carry state, city and a region grouping. An order route is a customer-state and seller-state pair, exposed in agg_delivery_by_route.	Reporting only by customer state would blame the destination for problems originating with a distant seller.
"WHEN" means calendar time and trend	dim_date is a full conformed date dimension, and fct_orders is partitioned by order_purchase_date so month-over-month scans stay cheap.	Deriving months with string functions on a timestamp would prevent partition pruning and make trend queries scan the whole table.
Late delivery must be linked to satisfaction	Reviews are deduplicated to one per order in int_order_reviews and joined onto fct_orders at matching grain.	Joining raw reviews would duplicate orders and distort both the average review score and the revenue total.
Late delivery must be linked to money	Order revenue is aggregated from item grain into fct_orders once, in an intermediate model, and reconciled against staging.	Joining items to orders inside the analysis would multiply revenue by the item count.
A promise can be wrong even when delivery is fine	delay_vs_promise_days and total_delivery_days are stored as separate measures, never collapsed into one "late" flag.	A single boolean would hide the far cheaper fix: correcting the estimate rather than the logistics.

5. Repository Structure

One main branch, as the brief requires. Feature work happens on short-lived branches merged via pull request so CI runs before anything reaches main.

Path	Contents
README.md	Architecture, tool rationale, run order and reproduction steps.
requirements.txt	Pinned versions, Python 3.11.
.env.example	Variable names only, never values.
.gitignore	data/, .env, credential JSON, dbt target/, logs/.
Makefile	make setup manifest gate ingest build test analyse dashboard lint unit all.
warehouse.py	Shared BigQuery connection, imported by both the notebooks and the dashboard so the two cannot read different datasets.
docs/	This plan, the architecture and star-schema diagrams, the data dictionary, the metric dictionary, the report, collected test evidence, and ADR-001 to ADR-006.
ingestion/	load_olist.py, the explicit BigQuery schemas (autodetect disabled), the optional BrasilAPI enrichment, and manifest.json.
quality/	The Great Expectations data context, suites and the pre-load gate script.
dbt_project/	24 models across staging, intermediate and marts; 161 tests; 4 macros; 1 snapshot; packages.yml and profiles.yml.example.
notebooks/	01_profiling, 02_delivery_performance (the primary business case) and 03_business_kpis.
dashboard/	app.py - the Streamlit dashboard over the same marts models.
orchestration/	Dagster definitions: assets, job and daily schedule.
tests/	pytest for the ingestion helpers and reconciliation logic.
.github/workflows/	ci.yml (credential-free: lint, dbt parse, pytest, secret scan) and scheduled.yml (nightly dbt build).
presentation/	The 10-minute executive deck.

6. Ownership and Access Control

Five or more people sharing one GCP project is the most likely source of accidental damage in this assignment. The controls below are not bureaucracy; they are what stops two people from redefining the same table on the same afternoon.

6.1 Role assignment

Role	Owns	GCP access granted
Platform / Ingestion Owner	The GCP project, the GCS bucket, the olist_raw dataset, load_olist.py, the manifest and the reconciliation report	Project: roles/bigquery.jobUser, roles/storage.admin. Dataset Editor on olist_raw only.
Modeling Owner	dbt staging, intermediate and marts models; the star schema; the ERD	Project: roles/bigquery.jobUser. Dataset Editor on olist_staging, olist_marts, olist_snapshots.
Quality Owner	dbt tests, GX suites, the fault-injection demonstration, the quality matrix	Project: roles/bigquery.jobUser. Dataset Viewer everywhere; Editor on a personal dbt_<name> dev dataset.
Orchestration / DevOps Owner	Dagster assets and schedule, GitHub Actions workflows, secrets, the Makefile	Project: roles/bigquery.jobUser. Owns the least-privilege CI service account.
Analytics / Communications Owner	Notebooks, metric dictionary, report, executive deck, insight cards	Project: roles/bigquery.jobUser. Dataset Viewer on olist_marts, read-only.

6.2 Non-negotiable rules

- One person creates the raw dataset and performs the upload. Everyone else reads it. This prevents conflicting schema changes on the same table.
- Every member develops dbt into their own dbt_<name> dataset using a dev target. Only a passing dbt build promotes anything to olist_marts.
- Local development authenticates with user OAuth (gcloud auth application-default login) so every BigQuery job is attributable to a person. No service-account key is ever downloaded to a laptop.
- Exactly one service account exists, for GitHub Actions, with the minimum roles it needs. Its key lives only in GitHub Secrets.
- Git is the version-control system for SQL, YAML, Python and documentation. BigQuery tables have no version history: if it is not in Git, it can be lost.
- Never commit .env, any service-account JSON key, anything under data/, dbt target output, or notebook outputs containing credentials.

7. Manual Setup Steps

READ THIS FIRST. These are the steps that must be performed by a person in a browser or terminal; they cannot be automated away. Do them in order. Each step ends with a verification command, and you should not move on until it passes. The "who" column matters: several of these must be done once, by one person, not by everyone.

Step	What	Who	Time
A	Create the GCP project, enable billing and APIs, invite the team	Platform Owner only	about 15 min
B	Authenticate gcloud locally	Every member	about 5 min
C	Download the Olist dataset from Kaggle	Platform Owner only	about 5 min
D	Create the GCS bucket and the four BigQuery datasets	Platform Owner only	about 5 min
E	Create the Python environment	Every member	about 10 min
F	Run the first data load	Platform Owner, after Phase 3	about 10 min
G	Create the GitHub Actions service account and secrets	DevOps Owner, at Phase 7	about 10 min

Step A — Create the GCP project

Platform Owner only. Approximately 15 minutes.

1. Open <https://console.cloud.google.com> and sign in with the Google account that will own the project. Use an account you will still control at submission time.
2. **Enable billing.** In the left menu choose Billing, then "Link a billing account". A new Google Cloud account receives a 300 USD / 90-day free trial. Billing must be enabled for the BigQuery API to function, but the always-free tier (1 TB of query scan and 10 GB of storage per month) covers this entire project many times over, since Olist is about 120 MB. Expected real spend: zero.
3. **Create the project.** Click the project selector at the top of the console, then "New Project". Name it bigdata-m2-group7. Google appends digits to make a globally unique Project ID, for example bigdata-m2-group7-473012. Copy that Project ID exactly: it is not the same as the display name, and every member needs it.
4. **Enable the APIs.** Go to "APIs and Services", then "Enable APIs and Services". Search for and enable the BigQuery API, then do the same for the Cloud Storage API. Each takes about 30 seconds to activate.
5. **Invite the team.** Go to "IAM and Admin", then "IAM", click "Grant access", and add each teammate Google email address with the role BigQuery Job User (roles/bigquery.jobUser) at project level. This lets them run queries but not delete datasets. The finer dataset-level permissions from Section 6 are granted in Step D.

Verify:

```
gcloud projects describe <PROJECT_ID>
```

The output must include lifecycleState: ACTIVE. Then post the Project ID in the team chat so everyone can put it in their local .env file as GCP_PROJECT_ID.

Step B — Authenticate gcloud locally

Every member. Approximately 5 minutes. The gcloud CLI is already installed on the lead machine, but no account is currently logged in.

Run these three commands in order. The first two open a browser window; approve with the same Google account that was invited in Step A.

```
gcloud auth login
```

Grants the gcloud and bq command-line tools access to your account.

```
gcloud auth application-default login
```

Creates the Application Default Credentials file that the Python client library, dbt and SQLAlchemy all read. This is a separate credential from the one above, and skipping it is the single most common cause of "could not automatically determine credentials" errors later.

```
gcloud config set project <PROJECT_ID>
```

Verify:

```
bq ls
```

This must return without an authentication error. Immediately after Step D it will list the four datasets.

SECURITY. Do not create or download a service-account JSON key for local work. User OAuth makes every query attributable to a person, which is what the course access-control guidance requires. A leaked service-account key in a public repository is an immediate and serious finding.

Step C — Download the Olist dataset

Platform Owner only. Approximately 5 minutes.

1. Open <https://www.kaggle.com/datasets/olistbr/brazilian-ecommerce> and sign in to Kaggle. A free account is required to download.
2. Click Download. The archive is about 45 MB compressed, roughly 120 MB unzipped.
3. Alternatively, use the Kaggle CLI after placing your API token at `~/kaggle/kaggle.json` with permissions 600:

```
kaggle datasets download -d olistbr/brazilian-ecommerce -p data/raw --unzip
```

1. Unzip into `data/raw/` inside the project folder. The `data/` directory is in `.gitignore`, so raw CSV files are never committed. Other members download it themselves, or pull it from the GCS bucket after Step F.
2. Confirm all nine files are present and note the row counts you should expect:

File	Expected rows	Role in the model
olist_orders_dataset.csv	99,441	Order header and delivery milestones, feeds fct_orders
olist_order_items_dataset.csv	112,650	Item lines and money, feeds fct_order_items, the central fact
olist_order_payments_dataset.csv	103,886	Payment records, feeds fct_payments; aggregate before joining
olist_order_reviews_dataset.csv	99,224	Review scores, deduplicated into fct_orders
olist_customers_dataset.csv	99,441	customer_id to customer_unique_id bridge, feeds dim_customer
olist_products_dataset.csv	32,951	Product attributes, feeds dim_product
olist_sellers_dataset.csv	3,095	Seller attributes, feeds dim_seller
olist_geolocation_dataset.csv	1,000,163	Zip-prefix coordinates; must be collapsed before use

File	Expected rows	Role in the model
product_category_name_translation.csv	71	Portuguese to English category names

Also record, for the data dictionary: the licence (CC BY-NC-SA 4.0), the download date, and the fact that the data covers September 2016 to October 2018 with sparse coverage at both ends.

Step D — Create the bucket and datasets

Platform Owner only. Approximately 5 minutes.

IRREVERSIBLE. A BigQuery dataset location cannot be changed after creation, and a dataset in one region cannot be joined to a dataset in another. We are using US multi-region for everything. The whole team must agree before this step runs; fixing it later means deleting and reloading.

Create the landing bucket:

```
gcloud storage buckets create gs://<PROJECT_ID>-olist-raw \
  --location=US \
  --uniform-bucket-level-access
```

Create the four datasets:

```
for ds in olist_raw olist_staging olist_marts olist_snapshots; do
  bq --location=US mk --dataset "<PROJECT_ID>:$ds"
done
```

Why four datasets rather than one: raw stays immutable so a defect remains reproducible; staging holds work in progress; marts is the only thing analysts and notebooks read; snapshots is isolated because dbt manages its history differently. Separating them also lets us give the Analytics Owner read-only access to marts alone.

Each member then creates a personal development dataset:

```
bq --location=US mk --dataset "<PROJECT_ID>:dbt_<yourname>"
```

Finally, grant the dataset-level roles from Section 6.1. In BigQuery Studio, click a dataset, choose Sharing, then Permissions, then "Add principal", and assign BigQuery Data Editor or BigQuery Data Viewer as appropriate.

Verify:

```
bq ls
bq show --format=prettyjson <PROJECT_ID>:olist_raw | grep location
```

The second command must print "location": "US". If it prints anything else, delete the dataset and recreate it now, before any data is loaded.

Step E — Create the Python environment

Every member. Approximately 10 minutes.

PYTHON VERSION. The default python3 on the lead machine is 3.14, which dbt does not support. The project pins Python 3.11. The existing course conda environment "elt" already contains dbt-bigquery, Great Expectations, pandas and SQLAlchemy and can serve as a fallback, but we create a dedicated environment so the repository reproduces from empty, which the acceptance checklist requires.

```
conda create -y -n bd-m2-g7 python=3.11
conda activate bd-m2-g7
pip install -r requirements.txt
```

requirements.txt will pin, at minimum:

```
dbt-core==1.11.*          dbt-bigquery==1.10.*
google-cloud-bigquery      google-cloud-storage
sqlalchemy>=2.0           sqlalchemy-bigquery    db-dtypes
pandas                    pyarrow                matplotlib
great-expectations==1.22.* # >=1.9: 1.8 pins altair<5, clashing
                           # with the altair>=5 Streamlit needs
dagster                   dagster-dbt            dagster-webserver
requests                  python-dotenv
pytest                    sqlfluff
```

Then copy .env.example to .env and fill in your Project ID, bucket name and dev dataset name. .env is git-ignored; .env.example contains variable names only and is committed.

Verify:

```
make debug
```

All connection checks must pass against your personal dbt_<name> dataset. Use make debug rather than a bare dbt debug: the Makefile is the only thing that loads and exports .env, so a direct dbt command fails with "Env var required but not provided".

Step F — Run the first data load

Platform Owner, once the Phase 3 code exists. Approximately 10 minutes for the first run.

```
make ingest
```

This single command computes a manifest (file name, byte size, sha256, row count) for all nine CSVs; uploads them to the GCS landing prefix; runs the Great Expectations file gate; loads each file into olist_raw with an explicit schema and no autodetect; attaches _batch_id, _loaded_at and _source_uri metadata columns; and prints a reconciliation table comparing source row counts with BigQuery row counts.

The load uses WRITE_TRUNCATE per batch, so running it twice is safe and produces identical row counts. Demonstrating that is itself a graded piece of evidence: idempotency.

Verify:

```
make ingest          # run it a second time
bq query --use_legacy_sql=false \
  "SELECT COUNT(*) FROM `<PROJECT_ID>.olist_raw.orders`"
```

The count must still be 99,441 after the second run. Commit the reconciliation log to docs/evidence/.

Step G — GitHub Actions service account

DevOps Owner, at Phase 7 only. Approximately 10 minutes.

1. IAM and Admin, then Service Accounts, then Create service account. Name it gh-actions-dbt.
2. Grant it BigQuery Job User at project level, and BigQuery Data Editor on olist_staging and olist_marts only. Do not grant Owner, Editor or project-wide Data Editor.
3. Open the service account, then Keys, then Add key, then Create new key, then JSON. The file downloads once.
4. In GitHub: repository, then Settings, then Secrets and variables, then Actions, then New repository secret. Create GCP_SA_KEY containing the entire JSON file contents, and GCP_PROJECT_ID containing the project ID.
5. Delete the downloaded JSON file from your laptop and empty the trash. It must never enter the repository, a chat message, or a screenshot.

IF A KEY LEAKS. Disable the key in the console immediately (Service Accounts, Keys, delete), create a new one, and update the GitHub secret. Removing the file in a later commit does not remove it from Git history: the key must be revoked, not just deleted.

8. Phased Delivery Plan

Day numbers are relative; map them to the actual submission deadline at the review meeting. A phase does not start until the previous phase Definition of Done is met and merged to main. The Definition of Done column is the acceptance test: it is deliberately written as something you can check, not something you can claim.

Phase	Days	Content	Definition of Done	Owner
0. Setup	D1	Manual Steps A to E. Repository scaffold, .gitignore, .env.example, requirements.txt, Makefile skeleton.	Every member can run bq ls and dbt debug successfully against their own dev dataset.	Platform + DevOps
1. Scope and design	D1-D2	Three business questions; KPI definitions with numerator, denominator and exclusions; grain statements; star-schema sketch; ADR-001 to ADR-006.	docs/metric_dictionary.md and docs/adr/ merged to main and read by everyone.	All, Modeling leads
2. Profiling	D2-D3	01_profiling.ipynb: row and column counts, types, null rates, uniqueness, date ranges, join cardinality, and each of the four Olist traps quantified with a number.	docs/data_dictionary.md complete; a written anomaly list; the four traps confirmed with evidence.	Quality + Analytics
3. Raw load	D3-D4	ingestion/load_olist.py, explicit JSON schemas, manifest, GCS upload, metadata columns, reconciliation report.	make ingest run twice produces identical row counts; load log committed to docs/evidence/.	Platform
4. Staging and intermediate	D4-D6	stg_* models (rename, cast, dedupe, translate, geo-collapse); int_order_payments, int_order_reviews, int_order_milestones.	dbt build --select staging intermediate is green; the fan-out check query returns zero extra rows.	Modeling
5. Marts	D6-D8	dim_customer, dim_product, dim_seller, dim_date; fct_order_items, fct_payments, fct_orders; agg_delivery_by_route, agg_delivery_monthly and the three KPI aggregates; partitioning, clustering, product snapshot.	Full dbt build green; the ERD matches the deployed tables; all grain tests pass.	Modeling
6. Quality	D8-D9	The complete test matrix from Section 9, the GX raw suite and checkpoint, and the fault-injection demonstration.	dbt and GX artifacts saved to docs/evidence/; the injected fault shows red, the fix shows green, both captured.	Quality
7. Orchestration and CI	D9-D10	Dagster assets (raw, GX gate, dbt, exports), daily schedule, retry policy, fail-stop demo. GitHub Actions ci.yml and scheduled.yml. Manual Step G.	A run where a failed upstream asset visibly skips downstream assets; a green CI check on a pull request.	DevOps

Phase	Days	Content	Definition of Done	Owner
8. Analysis	D10-D12	02_delivery_performance.ipynb (the primary business case: where, when, cost); 03_business_kpis.ipynb (monthly sales, top products, RFM); the BigQuery cost benchmark; insight cards. Then the Streamlit dashboard (dashboard/app.py) over the same marts models.	Notebooks read from olist_marts via SQLAlchemy with no manual CSV; each insight carries evidence, interpretation, action, expected impact and caveat; the dashboard runs end to end and its live reconciliation tab passes.	Analytics
9. Docs and deck	D12-D13	Architecture diagram, star-schema diagram, docs/report.md, the 10-minute executive deck.	The report answers why these tools, why this schema, and what could go wrong, not just what was built.	All
10. Dry run	D14	Clean-clone reproduction test, timed 10-minute rehearsal, Q&A preparation.	A teammate reproduces the pipeline from an empty folder using only the README; every member can defend at least one trade-off unaided.	All

9. Data Quality Matrix

Each rule below corresponds to a business risk, not just a technical property. Severity determines behaviour: an error blocks publication, a warning is published with an explanation. Nothing fails silently and nothing is silently coerced. The final three rules exist specifically because the headline number of this project is a late-delivery rate, and a late-delivery rate is trivially easy to compute wrongly.

Object	Rule	Tool	Severity	Action on failure
Raw files	All nine expected files present; column set and order match; sha256 and row count match the manifest	GX	error	Stop before load. Preserve the source file and the failure log.
Raw files	price and freight_value parse as numeric and are not null	GX	error	Stop before load; investigate the export.
stg_orders	order_id unique and not null; order_status within the eight allowed values	dbt	error	Block all mart models.
stg_order_items	order_id plus order_item_id unique; price >= 0; freight_value >= 0	dbt	error	Block all mart models.
stg_order_reviews	One row per order_id after deduplication	dbt	error	Fix the dedupe logic before fct_orders builds.
dim_customer	customer_key unique and not null; exactly one row per customer_unique_id	dbt	error	Block fact models: RFM would be wrong.
fct_order_items	order_item_key unique and not null	dbt	error	Block publication.
fct_order_items	customer_key, product_key and seller_key all resolve to their dimensions	dbt	error	Quarantine orphan rows and investigate the source.
fct_orders	purchase <= approved <= delivered to carrier <= delivered to customer	dbt singular	error	Return to staging and fix the cast or the filter.
fct_orders	order_payment_total is close to order_revenue plus order_freight within a documented tolerance	dbt singular	warn	Olist has known small mismatches from vouchers. Explain in the report; never coerce.
fct_orders (delivery)	total_delivery_days >= 0 and within a plausible range; carrier_transit_days >= 0; the stage durations sum to total_delivery_days	dbt_expectations	error	A negative or absurd duration means a milestone timestamp is wrong. Fix in staging before any delivery figure is published.
fct_orders (delivery)	Every order with status delivered has a non-null order_delivered_customer_date; is_late is null exactly when delivery date or estimate is null	dbt singular	error	Prevents silently counting undelivered orders as on-time, which would understate the late rate.
fct_orders (delivery)	is_late equals delay_vs_promise_days > 0 for every row, with no third state	dbt singular	error	The flag and the measure must never disagree; the headline late rate depends on it.
Reconciliation	sum(fct_order_items.item_price) equals sum(stg_order_items.price)	dbt singular	error	A fan-out bug exists. Fix the join before anything else.

Object	Rule	Tool	Severity	Action on failure
Reconciliation	count(distinct fct_orders.order_id) equals count(distinct stg_orders.order_id)	dbt singular	error	Investigate the dropped rows; an unintended filter is the usual cause.
Dates	All order_purchase_date values fall between 2016-09-01 and 2018-10-31	dbt_expectations	error	Investigate; a parsing or timezone error is likely.
Freshness	snapshot_date recorded on every load	documented	n/a	Static export: recorded as a stated limitation rather than alerted on.

9.1 The fault-injection demonstration (required)

Both course guides identify this as the strongest single piece of evidence available. It is a required Phase 6 deliverable, not an optional extra.

1. Run dbt build with the pipeline healthy and save the full passing output.
2. On a branch, inject two controlled faults into a copy of the staging data: a duplicated order_item_key, and a negative price.
3. Run dbt build again. Capture the failing tests, the failure counts, and the actual offending rows that dbt stores.
4. Show that the downstream mart models did not build, so the gate held.
5. Repair the data or the rule, rerun, and capture the green result.
6. In the report, state plainly which business misjudgement this gate prevented: duplicated keys inflate revenue, and negative prices corrupt every average.

10. Analysis Plan

Analysis A is the primary business case and gets the most depth and the headline chart. Analyses B, C and D are mandated by Section 5 of the brief and are each framed through the delivery lens so the report tells one coherent story. Each is specified with its exact definition, the documented trap it avoids, and the action it must produce. A chart without an action is not a finding.

Analysis	Exact definition	Trap avoided	Required output
A. Delivery performance: WHERE (primary)	From fct_orders, for delivered orders only: late rate = share of orders with delay_vs_promise_days > 0; median and p90 total_delivery_days; by customer_state, by seller_state, and by the customer-state / seller-state route pair. A minimum order volume floor suppresses noisy low-volume states.	Reporting only by destination hides origin-driven problems. Reporting a mean instead of a median lets a handful of extreme outliers define a state. Including undelivered orders makes late rates look artificially low.	A ranked list of the worst states and routes with the order volume and revenue behind each, and a specific recommendation on which carrier lane or seller region to address first.
A. Delivery performance: WHEN (primary)	Monthly late rate and median delivery days over the full window; day-of-week of purchase; decomposition of total_delivery_days into approval, carrier handover and transit stages to locate where the time is actually lost.	Attributing all delay to the carrier when the delay is in seller handover. Reading partial first and last months as a trend.	A stage-level statement of where the days go, whether performance improved over the period, and which months need extra capacity.
A. Delivery performance: COST (primary)	Mean review_score by lateness bucket; share of revenue on late orders; the same cut restricted within a single state and category to control for regional and product mix. Also: how often the estimated date itself is wrong, and by how much.	Confusing correlation with causation. Comparing regions with structurally different logistics. Treating a wrong promise and a slow delivery as the same problem.	A quantified statement of what lateness costs in satisfaction and exposed revenue, plus a proposed A/B or quasi-experiment to test the causal claim. Separate recommendations for fixing the estimate versus fixing the logistics.
B. Monthly sales trend (brief requirement)	Revenue = sum(item_price) from fct_order_items, grouped by month of order_purchase_date. Freight reported as a separate series, never merged into revenue. Order count = count(distinct order_id). Excludes canceled and unavailable statuses. Overlaid with the monthly late rate from Analysis A.	Never join payments to items, which multiplies revenue. Never mix order grain with item grain in one aggregate.	Seasonality read, months of growth and decline, and whether service quality degrades in peak months. Must annotate that Sep-Oct 2016 and Sep-Oct 2018 are partial months, not a collapse.
C. Top products (brief requirement)	Revenue, units sold and distinct orders reported together, by product and by English category name, with a minimum-sample floor. Each category also carries its late rate, median delivery days and mean review score.	Ranking on revenue alone flatters high-price, low-volume items and hides the real volume drivers.	Identification of categories that sell well but ship badly, which is the highest-value intersection of the two analyses, plus the usual assortment and supply recommendation.

Analysis	Exact definition	Trap avoided	Required output
D. Customer segmentation, RFM (brief requirement)	Aggregate to order grain first, then to customer_unique_id. Recency measured against the dataset snapshot date, never against today. Frequency = distinct orders. Monetary = sum of order revenue. Each segment cross-tabulated with its delivery experience.	Using customer_id splits one repeat buyer into several one-time customers and destroys the entire segmentation.	Named segments such as champions, at-risk, dormant and new, each with a retention or reactivation action, plus a quantified figure for how much high-value customer revenue is exposed to late delivery.

10.1 Insight card format

Every finding presented to executives uses the same five-part structure, so the audience always knows what is measured and what is inferred.

Element	What it must contain
Observation	What happened, with specific values, a period, and a comparison baseline.
Interpretation	The likely reason, with evidence and speculation clearly separated.
Decision	Who should do what, specifically.
Expected impact	Which KPI would move, and roughly by how much.
Caveat	Data coverage, causal limits, sample size, or known bias.

10.2 Optional enrichment, only after the MVP is green

The brief permits appending external data. If time allows after Phase 8, we will enrich dim_date with Brazilian public holidays from BrasilAPI, a free, unauthenticated, terms-permitting REST endpoint. This is a small, low-risk ingest that produces concrete evidence for course unit 2.4: request timeouts, retry with exponential backoff, status-code branching, raw-response archiving, and source metadata such as URL, fetch time and record count. It also makes the seasonality analysis materially better. It will not be started while any earlier phase is incomplete.

11. Orchestration and CI

11.1 Dagster

Dagster models the pipeline as assets, meaning the data produced, rather than as tasks, so lineage and materialisation status come for free. The asset graph:

asset 1	olist_source_files. Manifest plus the upload to the GCS landing prefix.
asset 2	raw_quality_gate. The Great Expectations checkpoint. Raising here stops every asset below it.
asset 3	olist_raw_tables. BigQuery load jobs and the row-count reconciliation.
assets 4+	dbt assets, via dagster-dbt. stg_* to int_* to dim_/fct_/agg_, with every dbt test surfacing as an asset check.
output	analytics_exports. The materialised KPI tables the notebooks and dashboard read.

- A daily schedule triggers the job. Retries are configured on the ingest assets only, because they are idempotent; transformation failures should surface, not silently retry.
- A failing quality gate stops downstream materialisation. We will capture a screenshot of exactly this, because a cron job that runs everything regardless of failure is not orchestration, and the course guide is explicit on this point.
- Each run records run_id, start and end time, input and output row counts, status and errors.

11.2 GitHub Actions

Workflow	Trigger	What it does
ci.yml	Every pull request and push to main	Installs pinned dependencies, runs sqlfluff lint on all dbt SQL, runs dbt parse to catch broken refs and macros, and runs pytest against the ingestion helpers. Needs no cloud credentials, so it is fast and safe.
scheduled.yml	Nightly cron, plus manual dispatch	Authenticates with the least-privilege service account from Step G and runs dbt build against olist_staging and olist_marts, uploading test results as a workflow artifact. This is the automated data-update and quality-check schedule the brief asks for.

11.3 Interactive dashboard (Streamlit)

The notebooks are the analyst's surface; they are not something a COO will open. The dashboard is the same analysis for someone who will not run Jupyter, and it is what gets demonstrated live during the presentation.

Tab	Answers	Notable content
Where	Which destinations and which routes are late?	Late rate by destination state against the national average; an origin-to-destination region heat map; the routes carrying the most late orders, with the revenue behind each.
When	Which months, and is it improving?	Order volume against late rate by purchase month; a first-half versus second-half comparison stated as a percentage-point change; the stage decomposition showing whether days are lost in payment approval, seller handover or carrier transit.

Tab	Answers	Notable content
What it costs	What does lateness cost in satisfaction and revenue?	Review score by lateness bucket; the same penalty recomputed within each state as a control; revenue exposed to late delivery by region; and whether the promise itself is padded or optimistic.
Business KPIs	The three metrics the brief mandates	Monthly sales with the late rate overlaid; a revenue-versus-late-rate scatter that surfaces categories which sell well but ship badly; RFM segments with their delivery experience attached.
Data quality	Can these numbers be trusted?	Runs the revenue reconciliation, the order-grain check, the person-grain RFM check and an app-versus-model agreement check LIVE against the warehouse, then states plainly what each of the two pipeline gates blocks.

Two design rules, both inherited from the architecture rather than from Streamlit convention:

- **It reads the marts layer only.** No CSV, no raw tables, no local files. Every number comes from a dbt model that has tests attached, so the dashboard, the notebooks and the report cannot disagree with one another.
- **It does not redefine metrics.** Definitions live in the metric dictionary and are implemented in dbt. Interactive filtering does force the late rate to be recomputed against `fct_orders`, because a pre-aggregated model cannot be sliced by an arbitrary filter combination; that SQL is confined to a single constant in the app, and the Data quality tab asserts live that it still agrees with `agg_delivery_monthly`. The dashboard therefore cannot silently drift from the pipeline.
- The shared connection module lives at the repository root and is imported by both the notebooks and the dashboard, so the two cannot end up reading different datasets while both appear correct.
- Run with `make dashboard`. It opens on `localhost:8501` and needs the same user OAuth credentials as everything else; no service-account key is involved.

12. Coverage of the Assignment Brief

A direct mapping from each numbered requirement in Module 2 Assignment Project V2 to where it is satisfied in this plan. Use this table in the review to confirm nothing is missing.

Brief requirement	How this plan satisfies it	Where
1. Data ingestion	Olist, one of the three permitted datasets, loaded into BigQuery by a Python script with an explicit manifest, checksums, a GCS landing zone and reconciliation. Optional external enrichment via BrasilAPI.	Sections 3 and 7 Step F; Phase 3
2. Data warehouse design	Star schema with declared grain: three fact tables, four dimensions, plus aggregates. Implemented in BigQuery with partitioning and clustering.	Section 4; Phase 5
3. ELT pipeline	dbt-core transforms raw to staging to intermediate to marts, with cleaning, validation and derived measures: order revenue, customer lifetime value, delivery days, delay days.	Sections 3 and 4; Phases 4 and 5
4. Data quality testing	A seventeen-rule quality matrix across dbt tests and Great Expectations, covering nulls, duplicates, relationships, ranges, date logic, delivery-duration validity and source-to-mart reconciliation, plus a live fault-injection demonstration.	Section 9; Phase 6
5. Data analysis with Python	SQLAlchemy connection to BigQuery; pandas exploration; the primary delivery-performance analysis (where, when, cost), plus the three mandated metrics: monthly sales trends, top-selling products and RFM customer segmentation.	Section 10; Phase 8
6. Pipeline orchestration (optional)	Taken: a Dagster asset DAG with a daily schedule, retry policy and demonstrated fail-stop; GitHub Actions for pull-request CI and a nightly scheduled build.	Section 11; Phase 7
7. Documentation	Architecture and star-schema diagrams via draw.io or Excalidraw; a report explaining tool choices, schema-design justification and findings with charts; ADRs recording rejected alternatives; data and metric dictionaries.	Sections 2, 3.2, 5; Phase 9
8. Executive presentation	A 10-minute deck following the guide recommended time allocation, with an executive summary, business value, high-level architecture, three insights, risks and a roadmap, followed by 5 minutes of Q&A with all members presenting.	Section 13; Phases 9 and 10
Deliverable: GitHub repo	Single main branch, all code and documentation, reproducible from an empty environment, no credentials committed.	Sections 5 and 6.2; Phase 10
Deliverable: Jupyter notebooks	Three notebooks: profiling, delivery performance (primary), and business KPIs, all reading from the marts layer.	Section 10; Phases 2 and 8
Deliverable: Slide deck	Executive summary and key findings, executive-friendly visuals, ROI and KPI framing.	Section 13; Phase 9
Deliverable: Dashboard	A Streamlit application over the marts layer: five tabs (Where, When, What it costs, Business KPIs, Data quality) with month, region and minimum-volume filters. It reads only tested dbt models and never redefines a metric, and its Data quality tab re-runs the revenue, grain and person-grain reconciliations live against the warehouse.	Section 11.3; Phase 8

13. Executive Presentation Structure

Ten minutes presenting, five minutes of Q&A, a mixed technical and business audience, all members speaking.

Time	Content	The one message the audience must leave with
0:00-1:00	Problem and executive summary	Late deliveries are concentrated in specific routes and months, they measurably depress review scores, and we can now see exactly where.
1:00-2:20	Data and architecture	The pipeline is rerunnable, clearly layered, and a quality gate stops bad data before anyone sees it.
2:20-3:30	Model and definitions	Declared grain and reconciliation testing prevent revenue being inflated by joins and the late rate being diluted by undelivered orders.
3:30-7:30	The delivery findings: where, when, what it costs	Each one: evidence, then meaning, then the action. Nothing else.
7:30-8:40	Quality and risk	Here is a test failing on purpose, here is what it caught, here is what our data cannot tell you.
8:40-10:00	Recommendations and roadmap	These are the owners, the KPIs, and the next experiment.

13.1 Q&A preparation: every member must be able to answer

Likely question	Our answer
Why delivery performance as the business case?	It is the one question in this dataset that connects operations, customer satisfaction and revenue in a single chain, and every element of it is measurable from the source data rather than inferred. It also gives the schema a reason to exist: order grain, route geography and a conformed date dimension are all forced by it, not decorative.
How do you define "late"?	delay_vs_promise_days greater than zero, measured only on orders that actually reached the customer. We report the promise error and the actual transit time as separate measures, because a wrong promise and a slow delivery need different fixes.
Why Olist rather than the other datasets?	It has the richest set of related tables, so it demonstrates multi-table modelling, revenue reconciliation and customer segmentation. Instacart has no money columns; London Bicycles has no customer entity, which would make the required segmentation analysis impossible.
Why is the fact grain one order item?	Product and seller analysis requires item-level detail, and order_id alone is not unique at that level. Order-level metrics live in a separate fact so the two grains never mix.
How do you know the revenue figure is right?	Reconciliation tests assert that total revenue in the mart equals total revenue in staging. Payments are aggregated to order grain before any join, so fan-out cannot inflate the number.
Why not Spark or Kafka?	The data is 120 MB and static. A cluster or a broker would add operational cost with no benefit. We have documented the thresholds, namely data beyond single-node memory or a decision that loses value within minutes, that would justify each.
What does BigQuery cost you?	Effectively nothing: the free tier covers 1 TB scanned per month and this dataset is 120 MB. We also have a benchmark showing bytes scanned before and after partitioning and clustering.

Likely question	Our answer
What happens when a test fails?	Error-severity rules block publication and preserve the failing rows. Warnings publish with an explanation. We can show you a failure and the fix.
Are these findings causal?	No. They are descriptive and correlational. Where we suggest a causal mechanism we label it as a hypothesis and propose an A/B or quasi-experimental test.
Who can change the data?	One person owns raw uploads. Everyone else develops in an isolated dataset and promotes only through a passing build. Local access is user-attributed OAuth; the only service account is for CI and has the minimum permissions it needs.

14. Risks and Mitigations

Risk	Mitigation
Revenue is silently inflated by payment or review fan-out. This is the most likely way this project produces a confidently wrong number.	Grain declared before any SQL; payments and reviews aggregated in <code>int_models</code> ; reconciliation tests at error severity so the build fails rather than publishing.
Team members overwrite each other BigQuery tables.	Per-person <code>dbt_<name></code> development datasets; only a passing build promotes to <code>olist_marts</code> ; a single named owner for raw uploads.
A service-account key or <code>.env</code> file is committed to a public repository.	<code>.gitignore</code> written in the first commit; <code>.env.example</code> holds names only; a secret scan in CI; the CI key exists only in GitHub Secrets and is revoked, not merely deleted, if exposed.
Scope creep into Kafka, Spark or Redis in order to look advanced.	Explicitly out of scope in Section 3.2, with the adoption threshold for each documented in an ADR. Bonus work starts only after the MVP is green.
Region mismatch between GCS and BigQuery.	Both pinned to US in Step D; <code>load_olist.py</code> asserts the dataset location at runtime and fails fast.
dbt fails to install on the default Python 3.14.	A dedicated conda environment pinned to Python 3.11, documented in Step E and in the README.
Partial first and last months are misread as a business collapse.	Annotated in the notebook, the report and the deck; the trend chart either trims or visually marks those months.
The team runs out of time before analysis and the deck.	The phase gates front-load the pipeline; Phases 8 to 10 are protected. Optional enrichment and any bonus work are explicitly last.
Nobody but the author can run the pipeline at submission time.	Phase 10 requires a different team member to reproduce the whole pipeline from a clean clone using only the README.

15. Final Acceptance Checklist

The project is finished when every line below is true. This is the same checklist we will run in Phase 10.

- [] From a clean clone with empty BigQuery datasets, make all runs ingest, then the GX gate, then dbt build, then pytest, then the notebooks, and every stage passes.
- [] make ingest run twice produces identical row counts, so idempotency is proven rather than asserted.
- [] dbt build reports zero failures, and dbt docs generate renders the complete lineage graph.
- [] The fault-injection branch reproduces a red test, and the fix turns it green, both captured in docs/evidence/.
- [] A Dagster run with a deliberately failing upstream asset shows downstream assets skipped.
- [] A GitHub Actions check is green on a pull request.
- [] All three notebooks execute top to bottom reading from olist_marts, with no manual file upload anywhere.
- [] The partition and cluster benchmark table, showing bytes scanned and duration for all three variants, is recorded in the report.
- [] Every table has a documented grain, primary key, owner and description.
- [] The primary delivery-performance analysis answers where, when and at what cost; the three brief-mandated analyses are complete; each finding has an action and a caveat.
- [] The architecture diagram, star-schema diagram, data dictionary and metric dictionary are mutually consistent.
- [] git ls-files shows no .env, no service-account key, and nothing from data/.
- [] The presentation fits in 10 minutes when timed, and every member can explain at least one key trade-off unaided.

16. Review and Sign-off

Development begins only after this section is complete. Reviewers should focus on Section 2 (locked decisions), Section 4 (grain and fan-out), Section 6 (who owns what) and Section 8 (schedule): those are the choices that are expensive to reverse.

Name	Role from Section 6	Date	Approved / comments
	Platform / Ingestion Owner		
	Modeling Owner		
	Quality Owner		
	Orchestration / DevOps Owner		
	Analytics / Communications Owner		

Open questions to settle at the review meeting:

- The actual submission deadline, so the relative day numbers in Section 8 can be mapped to dates.
- Who takes each of the five roles in Section 6.
- Whose Google account owns the GCP project and the billing account.
- Whether the optional BrasilAPI holiday enrichment is in scope, given the deadline.